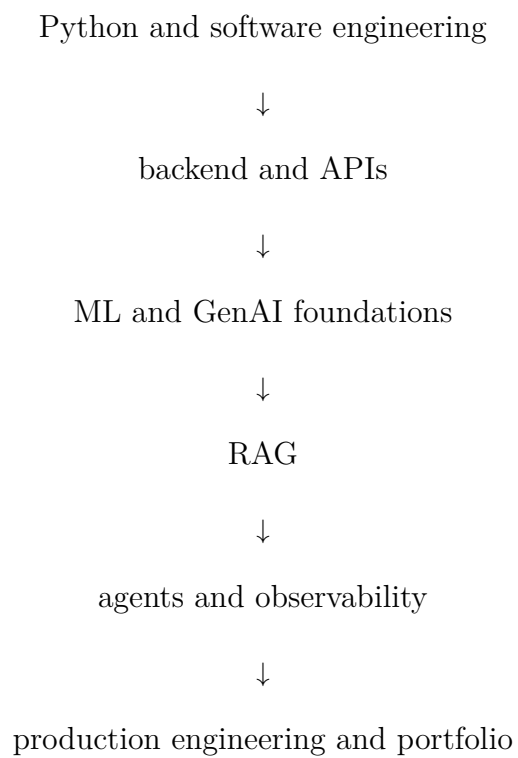


GenAI Developer Roadmap

Roadmap focus



Phase 0: Environment Setup (Days 1-3)

Goal: Professional developer environment ready. No coding yet - just tooling.

Install

- Python 3.12+ (use pyenv)
- VS Code with extensions: Python, Pylance, autoDocstring, GitLens, Docker
- Git (configure user.name and user.email)
- Docker Desktop
- PostgreSQL (local or Docker)
- Jupyter Notebook / Lab
- uv (modern Python package manager)

Accounts to create

- GitHub
- Hugging Face
- OpenAI
- Anthropic
- Google Colab
- Pinecone or Qdrant

Additional setup

- Set up a personal blog on Dev.to or Hashnode and document the journey from Day 1.

Month 1: Python Mastery + Software Engineering

Goal: Become a competent Python developer who writes production-grade code.

Week 1-2: Python Deep Dive

- Core Python: variables, functions, OOP (classes, inheritance), decorators, generators, iterators, context managers.
- File handling, modules/packages, JSON, and CSV.
- Professional Python: type hints (mypy), virtual environments with uv, exception handling, and logging.
- pytest basics - write tests from Day 1; every project should have a tests/ directory.
- Data structures: arrays, lists, dicts, sets, stacks, and queues at a practical level.

Week 3: Git Mastery + CI/CD Basics

- Repository management, branching strategies, pull requests.
- Resolving merge conflicts and rebasing.
- GitHub Actions - create a first CI pipeline that runs pytest on every push.

Week 4: SQL + NumPy / Pandas

- SQL: SELECT, WHERE, JOIN (INNER, LEFT), GROUP BY, aggregations, subqueries, indexes.
- PostgreSQL: table creation, relationships, and basic normalization.
- NumPy: vector operations, broadcasting, and array manipulation.
- Pandas: loading data, cleaning, filtering, transformations, groupby, merging datasets, and handling missing values.

Month 1 Project: Data Processing CLI Tool with Tests

- Read a CSV file containing sales or customer data.
- Clean and validate the data using Pandas.
- Store structured results in PostgreSQL.
- Generate a summary report.
- Include a pytest suite with >80% coverage, type hints, proper logging, and a GitHub Actions CI pipeline.

Month 2: Backend Engineering + APIs

Goal: Build production-ready backends that serve AI applications.

Week 1-2: FastAPI Mastery

- Routing, path/query parameters, request/response models.
- Pydantic v2 validation.
- Dependency injection system.
- Authentication: JWT tokens and OAuth2 password flow.
- Error handling, custom exception handlers, middleware, and background tasks.
- File upload handling for document processing.

Week 3: Databases + ORM

- PostgreSQL: advanced queries, table relationships, and indexing for performance.
- SQLAlchemy 2.0 (async version) and Alembic migrations.
- Redis basics: caching, rate limiting, and session storage.

Week 4: API Architecture + Deployment Prep

- REST principles, HTTP methods, status codes, and JSON payloads.
- API security: CORS, security headers, rate limiting, and input validation.
- Docker: production Dockerfile for FastAPI (multi-stage build) and Docker Compose for API + PostgreSQL + Redis.

Month 2 Project: Document Management API (Fully Dockerized)

- FastAPI + PostgreSQL + SQLAlchemy + Redis caching layer.
- User registration/login (JWT), file upload with metadata storage, search/filter endpoints, and rate limiting.
- Dockerfile + docker-compose.yml, pytest suite, GitHub Actions CI/CD pipeline, and setup documentation.

Month 3: ML Foundations + Generative AI Basics

Goal: Understand how AI works and get the first hands-on experience with LLMs.

Week 1: Practical Math (Lightning Fast)

- Linear algebra: vectors, matrices, dot products (intuition only, not proofs).
- Statistics: mean, variance, standard deviation, probability basics, normal distribution.
- Optimization: gradient descent intuition.
- Resource: 3Blue1Brown "Essence of Linear Algebra" and "Neural Networks" series at 2x speed.

Week 2: Classical ML

- Features vs labels, train/validation/test split, overfitting vs underfitting, cross-validation.
- Logistic Regression, Decision Trees, Random Forest, XGBoost.
- Metrics: accuracy, precision, recall, F1-score, RMSE.
- Tool: Scikit-learn.
- Project: customer churn prediction - data -> model -> evaluation -> FastAPI prediction endpoint.

Week 3: Deep Learning Basics (PyTorch)

- Neural-network building blocks: layers, activation functions, and loss functions.
- Backpropagation intuition.
- Build a simple MNIST handwritten digit classifier in PyTorch.

Week 4: LLM Fundamentals

- Tokens/tokenization, context windows, temperature, and top-p sampling.
- OpenAI API: chat completions, structured outputs, and function calling.
- Anthropic API: Claude features.
- Prompt engineering: system vs user prompts and few-shot prompting.
- AI security: prompt injection, jailbreak attempts, data leakage; defense through filtering, validation, and permissions.
- Cost optimization: token counting, model selection, response caching, and context-window management.

Month 3 Project: ML Prediction API

- Extend the Month 2 Document Management API with ML prediction endpoints.
- Maintain a separate PyTorch MNIST classifier repo with training, evaluation, and inference.

Month 4: RAG Systems (The Core Skill)

Goal: Build enterprise-grade retrieval systems.

Week 1: Embeddings + Vector Databases

- Embeddings as high-dimensional semantic representations.
- Similarity search: cosine similarity and Euclidean distance.
- Vector DB options: Pinecone or Qdrant.
- Open-source embedding models: sentence-transformers, BGE, and E5.
- Decision in the source roadmap: Qdrant for self-hosting; Pinecone for managed use.

Week 2: Document Processing + Chunking

- PDF extraction: PyPDF2, pdfplumber, unstructured.io.
- OCR basics with pytesseract for image-based PDFs.
- Chunking strategies: fixed-size, recursive character splitting, semantic chunking, and overlap handling.
- Metadata extraction: document title, section headers, and page numbers.

Week 3: Advanced RAG

- Retrieval + re-ranking: vector search followed by cross-encoder reranking.
- Hybrid search: vector similarity + keyword matching (BM25).
- Citations and source attribution.
- RAGAS evaluation: context precision, context recall, faithfulness, and answer relevance.

Week 4: Integration + Frontend Basics

- Connect the full RAG pipeline to the FastAPI backend.
- React basics, Next.js routing, and Vercel AI SDK.
- Build a chat UI that streams responses with citation highlights.

Month 4 Project: Production RAG Platform

- Frontend: Next.js chat interface with streaming responses.
- Backend: FastAPI + Qdrant/Pinecone + OpenAI/Anthropic API.
- Pipeline: PDF upload -> text extraction -> chunking -> embedding -> vector store.
- Features: semantic search, cited answers, and conversation history.
- Include RAGAS evaluation scores, LangSmith trace screenshots, and Docker deployment.

Month 5: AI Agents + Observability

Goal: Build multi-step reasoning systems with full visibility and control.

Week 1-2: Agent Architecture

- Tool calling: calculators, web search, APIs, and databases.
- Memory types: short-term, long-term, and entity memory.
- State management, conditional routing, and human-in-the-loop.
- Framework: LangGraph.
- Exposure only: CrewAI; master LangGraph first.
- Build 2-3 simple agents before the complex one: calculator, search, then combined.

Week 3: Observability

- LangSmith: trace prompts, tool calls, LLM output, and latency.
- Alternatives: Arize Phoenix or Weave.
- Debugging workflows for agent loops.
- Cost tracking and latency percentiles (p50, p95, p99).
- Error handling and graceful degradation when tools fail.

Week 4: Evaluation + Condensed Fine-tuning

- LLM-as-a-judge using a cost-conscious evaluator.
- Metrics: task completion rate, citation accuracy, latency, and cost per query.
- Fine-tuning with Unsloth on Google Colab free tier using 1B-3B models, not 8B models.
- Dataset target: 100-500 examples for a text-classification task.
- Backup plan from the source: switch to OpenAI fine-tuning after repeated Colab failures.

Month 5 Project: Observable Research Agent

- LangGraph agent that accepts a research topic.
- Web search tool, calculator tool, and document retrieval tool.
- Structured Markdown report synthesis.
- Full LangSmith tracing with shareable trace links.
- Simple evaluation pipeline for report quality.

Month 6: Production Engineering + Portfolio

Goal: Deploy the work and make the portfolio ready to present.

Week 1: Cloud Deployment

- AWS basics: EC2, S3, RDS, IAM.
- Alternative: Render.com or Railway.app for simpler deployment.
- SSL certificates, custom domains, and environment-variable management.
- Deploy the Month 4 RAG platform to a live URL.

Week 2: DevOps + System Design

- CI/CD: GitHub Actions -> build image -> push to registry -> deploy.
- AI system design: caching, queues, rate limiting, and horizontal scaling concepts.
- Monitoring with CloudWatch or basic uptime monitoring.

Week 3-4: Final Portfolio Polish

Project 1 - Enterprise RAG Platform

- Live URL with authentication, PDF upload, semantic search, cited answers, and streaming chat.
- README with architecture diagram, tech stack, RAGAS scores, and LangSmith screenshots.
- Blog post: "Building a Production RAG System: Lessons Learned".

Project 2 - Autonomous Research Agent

- Live URL with multi-tool agent and visible progress states.
- README with agent architecture, tool descriptions, and evaluation scores.
- Blog post: "LangGraph Agents: From Tutorial to Production".

Project 3 - ML + GenAI Hybrid System

- FastAPI backend serving both classical ML and a fine-tuned model.
- Separate PyTorch MNIST classifier repo.
- Demonstrate traditional ML, deep learning, fine-tuning, and deployment.

Portfolio essentials

- Personal portfolio website with project links, live demos, GitHub repos, and blog posts.
- At least 3 technical blog posts.
- One-page, project-focused, metrics-driven resume.
- Updated LinkedIn profile and project updates.

Final Skill Checklist

Programming

- Python (core + OOP + async)
- SQL (complex queries, optimization)
- Git (branching, PRs, rebasing)
- pytest (writing tests, fixtures, mocks)
- Type hints and static analysis

Backend

- FastAPI (routing, middleware, auth, background tasks)
- PostgreSQL (design, query, optimize)
- SQLAlchemy + Alembic (ORM + migrations)
- Redis (caching, rate limiting)
- Docker + Docker Compose
- CI/CD with GitHub Actions

Machine Learning

- Classical ML (scikit-learn pipeline)
- PyTorch basics (build and train a model)
- Evaluation metrics (precision, recall, F1, RMSE)

Generative AI

- LLM APIs (OpenAI, Anthropic)
- Prompt engineering
- RAG (embeddings, vector DB, chunking, reranking, citations)
- Agents (tool calling, memory, state management with LangGraph)
- Fine-tuning (LoRA/QLoRA or API-based)
- AI Security (prompt injection defense)

Production

- Observability (LangSmith tracing)
- Evaluation (RAGAS, LLM-as-a-judge)
- Cloud deployment (AWS EC2/S3/RDS or Render)
- System design (caching, queues, rate limiting, scaling concepts)
- Frontend basics (React + Next.js + streaming)

Career

- 3 production-grade projects on GitHub
- Technical blog with 3+ posts
- Portfolio website with live project links
- Updated LinkedIn and resume